

Automated Classification of White Blood Cell Subtypes Using CNNs

Fine-tuning ResNet-50 to tell four cell types apart

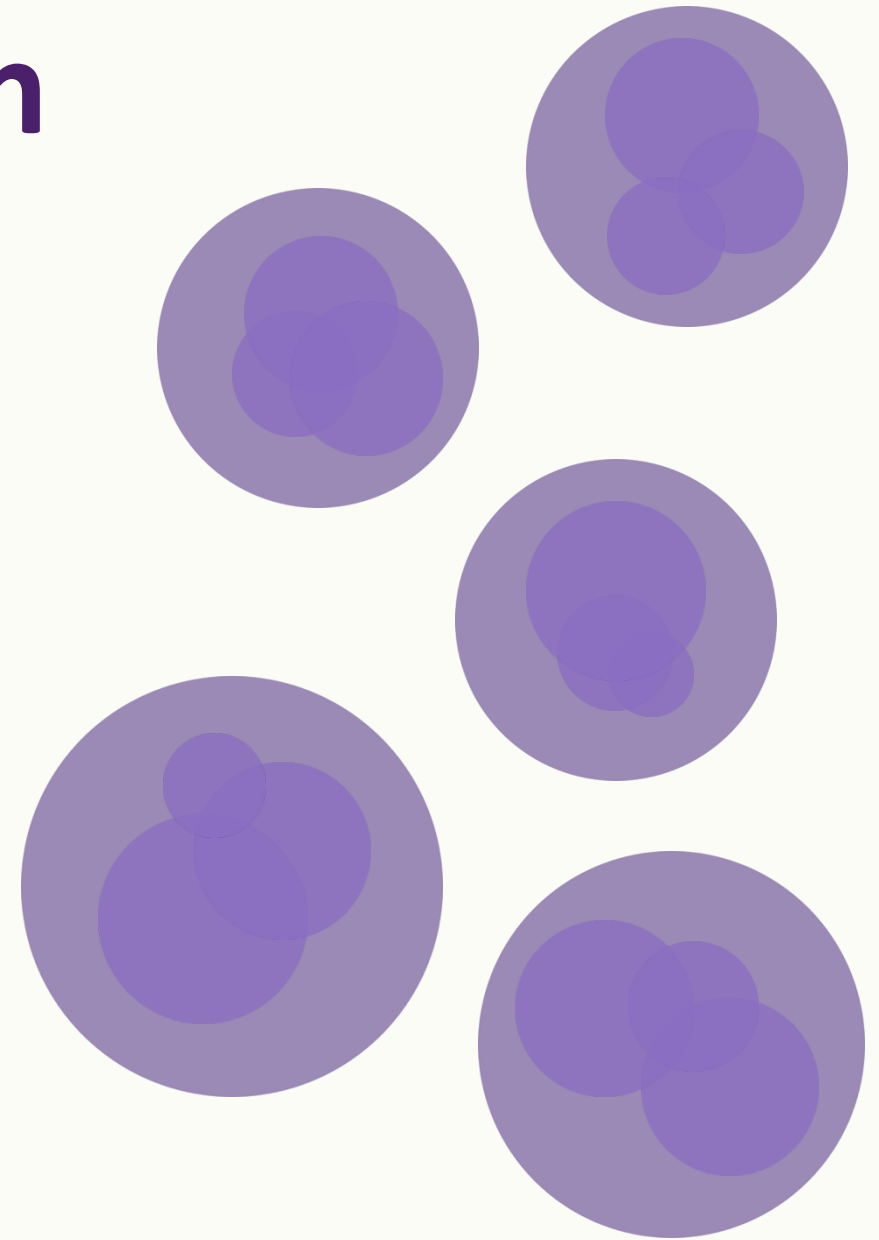
Group:

Yasmin Rocio Orduz Landazabal
Matthew Ragsdale

Instructor:

Dr. Meenalosini Vimal Cruz

CSCI 7090 – Advanced Computer Vision and Deep Learning
Midterm Presentation
Georgia Southern University
Fall 2026



The problem, our target metrics, and what's been tried

A differential count sorts white blood cells by subtype. It is among the most ordered tests for infection, allergic response and blood disorders, and it is still done largely by eye – after hundreds of cells per slide, borderline calls such as monocyte versus lymphocyte become error-prone.

Task & target metrics

Classify one microscope image as *eosinophil*, *lymphocyte*, *monocyte* or *neutrophil*.

Targets: Test accuracy $\geq 90\%$ (matches the published baseline below), reported with per-class precision/recall, macro-F1 and a confusion matrix – not one overall number.

Prior work reports one overall score

Praveen et al. (2021) – 90% subtype accuracy

YOLOv3 on this same Kaggle collection. Our images are already cropped to one cell, so this classification figure is the comparable baseline.

Davamani et al. (2025), Gupta and Mishra (2024)

Transfer-learning CNNs on related white-cell images, reporting one overall accuracy that hides which classes are confused.

Our angle: fine-tune ResNet-50 and report per-class numbers and a confusion matrix, not a second headline.

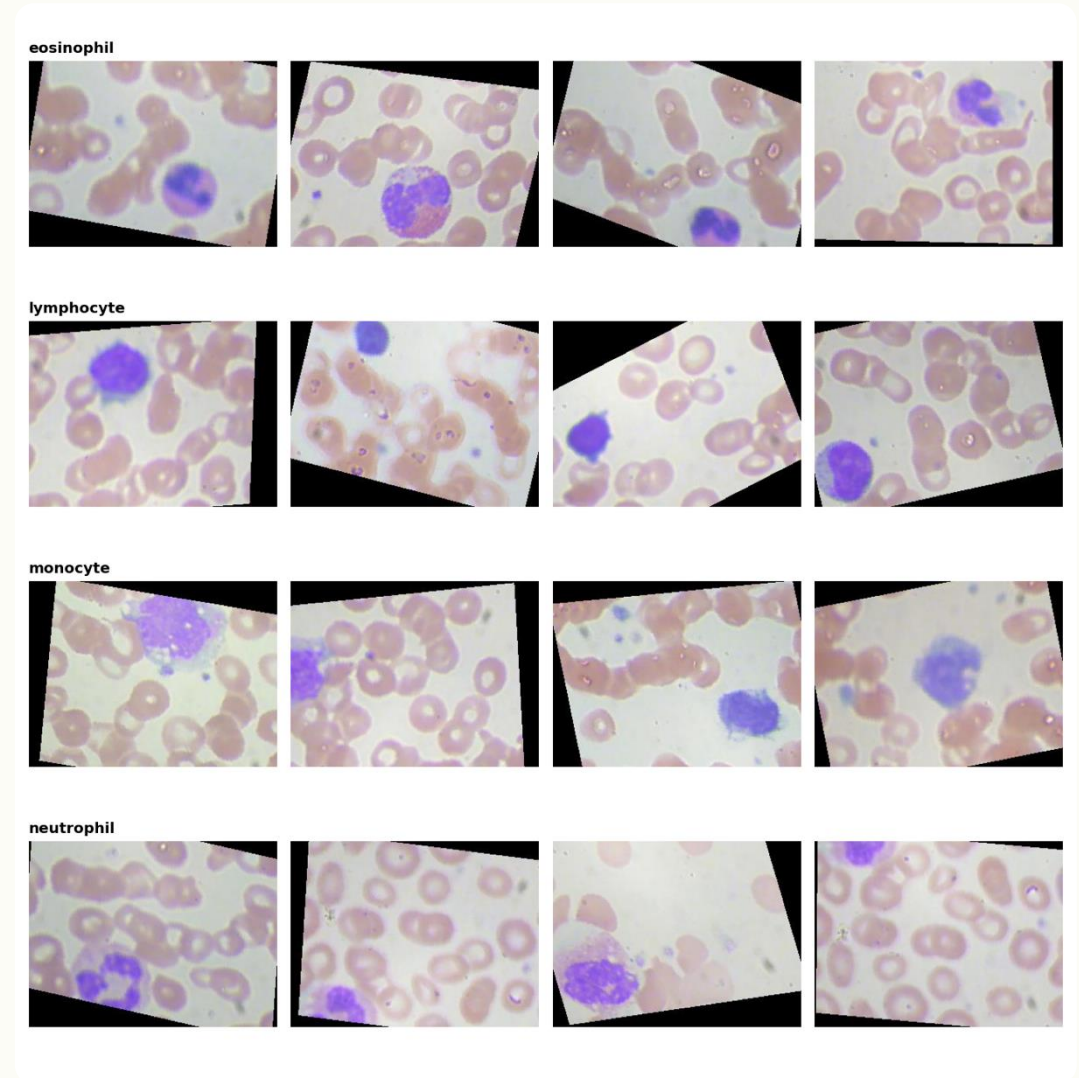
Dataset finalized, pipeline implemented

Class	Train	Test
Eosinophil	2,497	623
Lymphocyte	2,483	620
Monocyte	2,478	620
Neutrophil	2,499	624
Total	9,957	2,487

Kaggle, paultimothymooney/blood-cells, MIT license. Balanced classes (~25% each). 320×240 RGB JPEG throughout; 0 of 12,444 files corrupt.

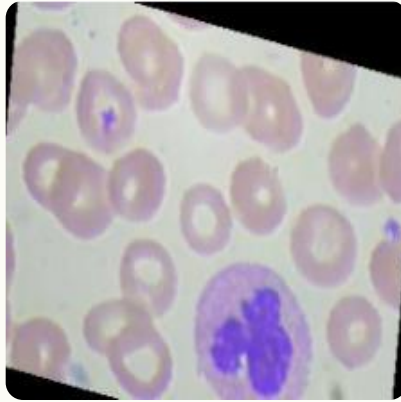
Implemented pipeline

Load → verify (0 corrupt) → resize to 224×224 → normalize (ImageNet mean/std) → light augmentation on training images only (flip, $\pm 10^\circ$ rotation, mild color jitter).

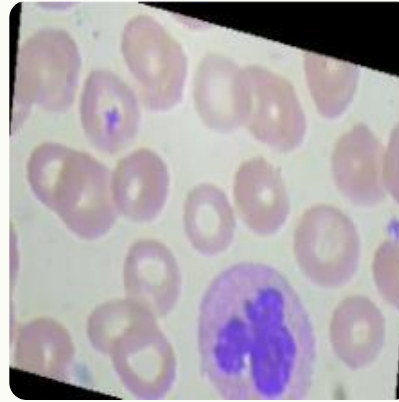


Random training images, one row per class.

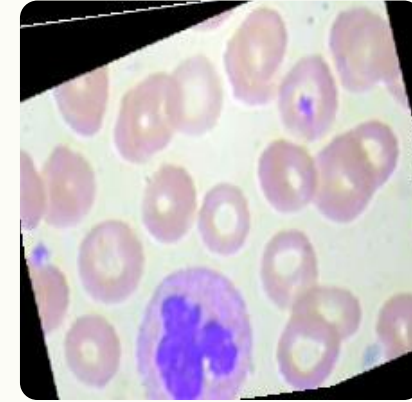
Every preprocessing choice, backed by evidence



1. Original (320x240)



2. Resized to 224x224



3. plus augmentation (train only)

Why 224x224: every one of the 12,444 images is the same 320x240; resizing to 224 exactly matches the size ResNet-50s ImageNet weights were trained on (He et al., 2016), avoiding a distribution shift between pretraining and fine-tuning.

Why light, not heavy, augmentation: Kaggle's 12,444 images are already augmented from a smaller raw set. Stacking heavy transforms on top of existing augmentation is a documented way to leak near-duplicate images across splits (Shorten & Khoshgoftaar, 2019) – confirmed by our own overlap check, next slide.

Ablation in progress: small-CNN runs with augmentation on vs. off, to quantify the effect directly on this dataset before the final presentation.

Data splitting: Order matters, and we checked for leaks

Ratios: train folder split 90/10 (stratified) into 8,961 train and 996 validation; 2,487 test images stay sealed. Overall 72% / 8% / 20%, chosen because the pool is large and balanced enough that 10% validation (996 images) already gives a stable estimate.

Stratified: sklearn `train_test_split(stratify=targets)` keeps each class near 25% in both train and validation, matching the original class balance.

Transform order: indices are split first, on raw file paths only; augmentation is then attached per-index, so it is applied after splitting, never before it, and only to the training indices.

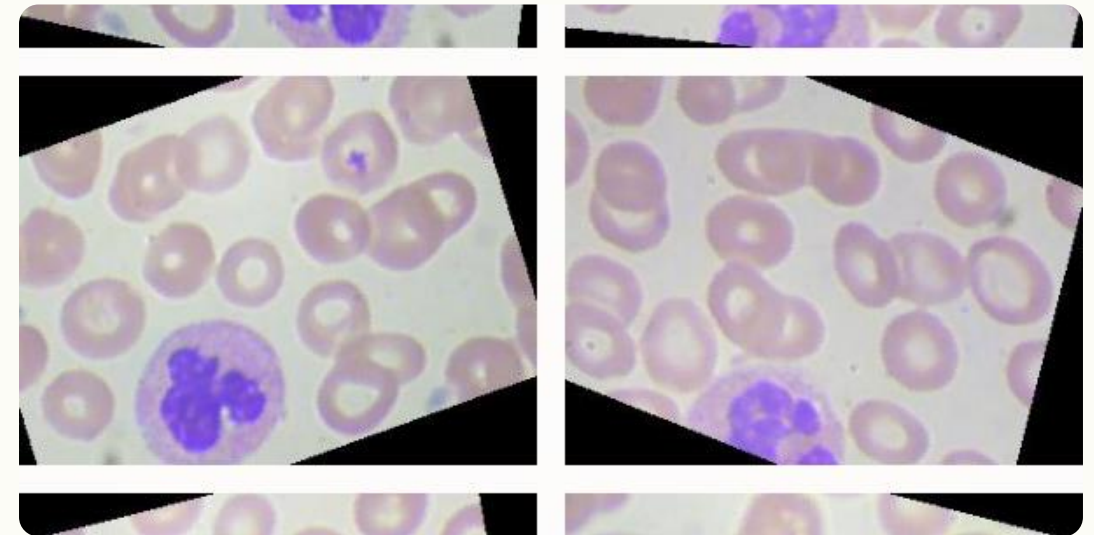
Leakage defense: the official test folder is opened exactly once, only by `evaluate.py`, after the model is chosen. We additionally embedded every image with an ImageNet ResNet-50 and searched for near-duplicate neighbours across splits.

25.4%

of validation images have a training neighbor at similarity greater than or equal to 0.95

0.5%

of test images do (highest 0.956, none reaches 0.98)



Most similar test versus train pairs found by the search. No near-identical pair (max 0.956); the risk is pre-existing in Kaggle's augmented set, not introduced by our split, and it affects validation more than the sealed test set.

Why ResNet-50, for this dataset and this budget

Candidate	Params	Status	Why chosen / not chosen, for our data and hardware
Small CNN (scratch)	0.39M	trained	Weak baseline, not a candidate for deployment. Establishes what transfer learning is worth on this exact dataset: 0.950 best val accuracy, noisy, 11 of 14 epochs to converge.
ResNet-50 (ImageNet)	23.5M	chosen	Fits our ~10K-image training set without the overfitting risk of a deeper net; residual connections (He et al., 2016) are proven for exactly this kind of transfer-learning task on white-cell images (Davamani, 2025; Gupta and Mishra, 2024). Trains to convergence in 8.4 minutes on one Colab T4. Best val accuracy 0.996-1.000 by epoch 4.
EfficientNet-B0	~5.3M	considered	Lighter, mobile-oriented (Tan & Le, 2019). Deprioritized: ResNet-50 already exceeds our 90% target with room to spare, so the extra tuning effort is not yet justified for this course project.
DenseNet-169	~14.1M	considered	Dense connections aid feature reuse (Huang et al., 2017). Deprioritized: similar accuracy expected to ResNet-50 on this task, at a smaller but still nontrivial GPU-time cost, without a clear benefit for a 4-class, single-cell problem.

Architecture is finalized as ResNet-50. Benchmarking EfficientNet-B0 and DenseNet-169 under an identical training budget is planned for the post-midterm ablation phase, not blind selection now.

Implementation: Head replacement, then fine-tune

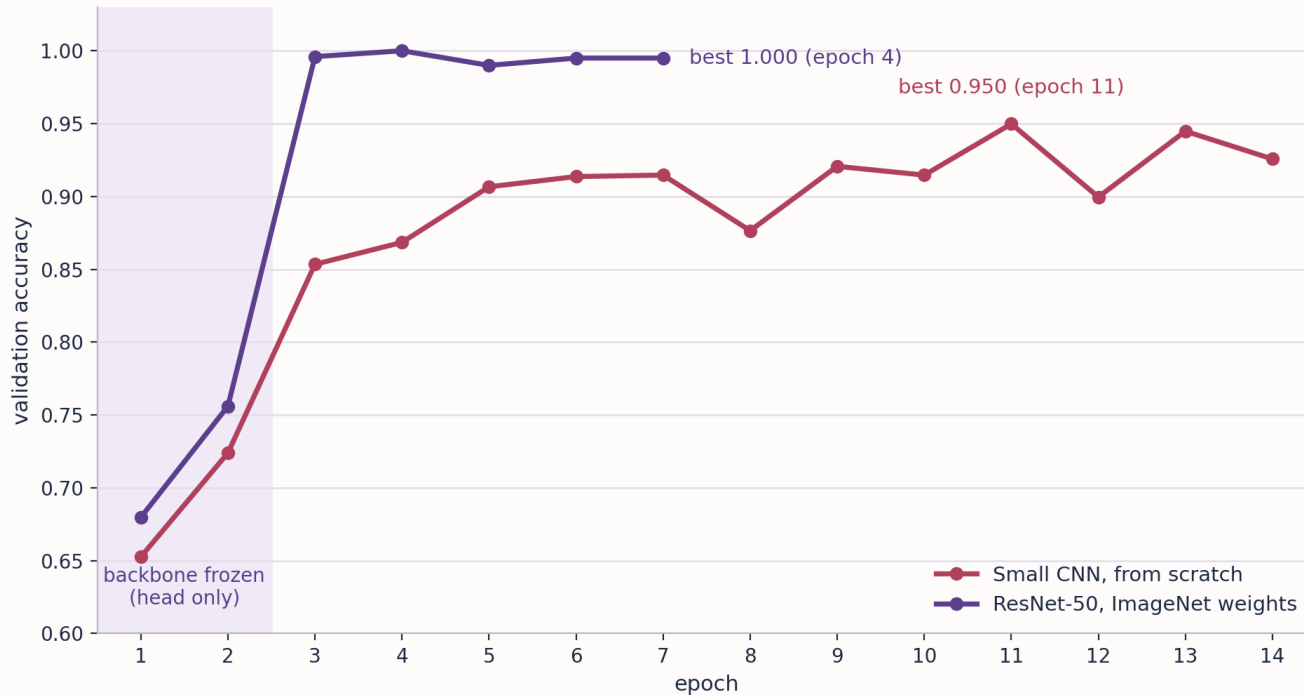
```
def build_model(name, num_classes, pretrained=True):  
    if name == "resnet50":  
        weights = ResNet50_Weights.IMAGENET1K_V2 if pretrained else None  
        model = resnet50(weights=weights)  
        model.fc = nn.Linear(model.fc.in_features, num_classes)  
        return model  
  
    def set_backbone_trainable(model, trainable):  
        for name, p in model.named_parameters():  
            is_head = name.startswith("fc.")  
            p.requires_grad = True if is_head else trainable
```

Two-stage training: epochs 1-2 train only the new 4-way head (8,196 params); from epoch 3 the whole network trains (23.5M params) at a 10x smaller learning rate for the backbone.

Optimizer: Adam, head lr 1e-3, backbone lr 1e-4 once unfrozen, weight decay 1e-4, batch size 64, mixed precision, seed 42.

Early stopping on validation loss (patience 3); best checkpoint kept. Runs on one Colab T4 GPU.

Fine-tuning beats training from scratch



1.000

ResNet-50, fine-tuned. Best at epoch 4; 0.995 at epoch 7

0.950

Small CNN from scratch. Best at epoch 11 of 14

0.756

ResNet-50 with the backbone frozen (epoch 2)

Validation split: 996 images from the train folder. One run per model, seed 42.

Validation is saturated, so the sealed test set decides

1

Validation images share the augmented train folder: 25.4% have a training neighbor at similarity ≥ 0.95 , versus 0.5% of test images. Every configuration lands near 100%, so validation can no longer tell them apart.

2

Test folder stays sealed until the configuration is final, then one evaluation only: Accuracy, per-class precision/recall, macro-F1, confusion matrix, next to the 90% baseline.

3

Before that: Augmentation on/off ablation, a rotation-aware overlap check, a full frozen-backbone run, and (budget permitting) EfficientNet-B0 and DenseNet-169 under the same training recipe.

Live demo: What the model sees

Prediction, class probabilities and Grad-CAM heatmap.

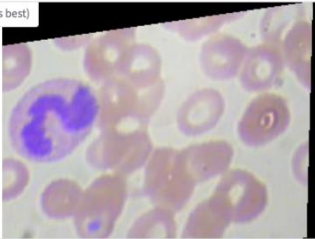
The heatmap lands on the white cell, not the red cells around it.

Note: Validation example, not a test result. Research prototype, not a clinical tool.

White blood cell classifier

Fine-tuned ResNet-50 that names the white blood cell type in a microscope image (eosinophil, lymphocyte, monocyte, neutrophil). The heatmap (Grad-CAM) shows which regions pushed the network towards its answer. **Research prototype, not a clinical tool.** Trained on the Kaggle Blood Cell Images data set.

Blood smear image (320x240 works best)

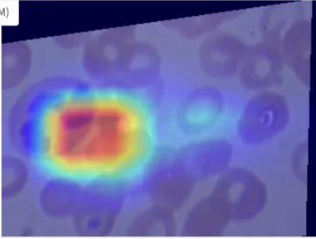


Predicted cell type

neutrophil

neutrophil	100%
eosinophil	0%
monocyte	0%
lymphocyte	0%

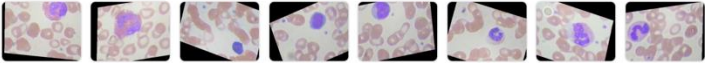
Where the network looks (Grad-CAM)



Clear

Submit

Examples



Runs · Use via API · Built with Gradio · Settings

References

- Davamani, K. A., Jawahar, M., Anbarasi, L. J., Ravi, V., Al Mazroa, A., & Robin, C. R. R. (2025). Deep transfer learning technique to detect white blood cell classification in regular clinical practice using histopathological images. *Multimedia Tools and Applications*, 84(9), 5699–5723. <https://doi.org/10.1007/s11042-024-19133-8>
- Gupta, S., & Mishra, A. (2024). Deep transfer learning-based classification of white blood cells using customized classification base. In *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing (IC3-2024)*. ACM. <https://doi.org/10.1145/3675888.3676117>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). IEEE. <https://doi.org/10.1109/CVPR.2016.90>
- Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4700–4708). IEEE. <https://doi.org/10.1109/CVPR.2017.243>
- Mooney, P. (2018). *Blood cell images* [Data set]. Kaggle. <https://www.kaggle.com/datasets/paultimothymooney/blood-cells>
- Praveen, N., Pun, N. S., Sonbhadra, S. K., Agarwal, S., Syafrullah, M., & Adiyarta, K. (2021). *White blood cell subtype detection and classification* (arXiv:2108.04614). arXiv. <https://arxiv.org/abs/2108.04614>
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 618–626). IEEE. <https://doi.org/10.1109/ICCV.2017.74>
- Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1), Article 60. <https://doi.org/10.1186/s40537-019-0197-0>
- Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning* (pp. 6105–6114). PMLR.

